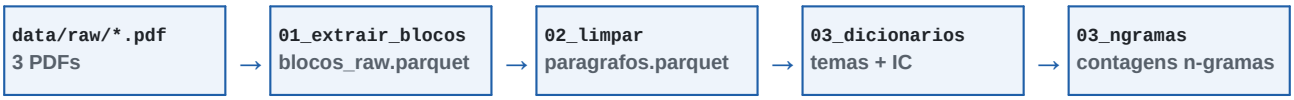
Do PDF às contagens: scripts 01 a 03

Como o projeto PDN Angola transforma os três planos nacionais de desenvolvimento em parágrafos limpos, frequências de temas e n-gramas · 05/11/2026

Visão geral

Os quatro scripts formam a primeira metade do pipeline. O 01 lê os PDFs sem alterar nada; o 02 decide o que é texto corrido e o que é ruído (tabelas, cabeçalhos, sumários); o 03_dicionarios e o 03_ngramas contam palavras sobre o mesmo corpus limpo e são independentes entre si. Todos rodam da raiz do projeto com `uv run python scripts/<nome>.py`.



Plano	Páginas	Blocos (01)	Corpo (págs.)	Parágrafos (02)	Títulos	Palavras
PND 2013–2017	234	4.743	13–234	3.144	312	64.305
PDN 2018–2022	308	8.693	21–306	3.544	615	114.502
PDN 2023–2027	209	3.546	7–209	2.516	381	59.743

Fontes: data/processed/blocos_raw.parquet e outputs/tables/resumo_limpeza.csv. Parágrafos e palavras excluem os títulos.

Funções partilhadas: comum.py

O 03_dicionarios importa de `scripts/comum.py` os caminhos, a semente aleatória (`SEMENTE = 20261105`), a lista de planos e duas funções: `carrega_paragrafos`, que lê o parquet e, por padrão, tira os títulos, e `carrega_dicionarios`, que transforma o YAML de temas em expressões regulares (secção 3). O 03_ngramas não importa o comum.py: tem a sua própria cópia das stopwords e da tokenização (ver Pontos de atenção).

1. 01_extrair_blocos.py — ler os PDFs

Entrada: data/raw/*.pdf · **Saída:** data/processed/blocos_raw.parquet

O script abre cada PDF com o PyMuPDF e pede o texto de cada página em formato de dicionário (`page.get_text("dict", sort=True)`). Nesse formato, a página é uma lista de *blocos* (parágrafos ou caixas de texto, como o PDF os desenhou), cada bloco tem *linhas* e cada linha tem *spans* (trechos com a mesma fonte). O script guarda uma linha por bloco, sem limpar nada.

A ideia central é guardar, além do texto, **como** ele aparece na página. Com a posição, o tamanho da fonte e o negrito, o passo 02 consegue reconhecer cabeçalhos, títulos e tabelas pela aparência, e não só pelo conteúdo.

```
            for b in d["blocks"]:
                if b.get("type") != 0:  # 0 = texto; 1 = imagem
                    continue
                textos, soma_tam, soma_neg, soma_car = [], 0.0, 0, 0
                n_girado = 0  # linhas que não correm da esquerda para a direita
                for ln in b["lines"]:
                    spans = ln["spans"]
                    t = "".join(s["text"] for s in spans)
                    if not t.strip():
                        continue
                    textos.append(t)
                    if ln["dir"][0] < 0.9:
                        n_girado += 1
                    for s in spans:
                        k = len(s["text"].strip())
                        soma_car += k
                        soma_tam += s["size"] * k
                        if (s["flags"] & 16) or ("bold" in s["font"].lower()):
                            soma_neg += k
                if not textos or soma_car == 0:
                    continue
```

scripts/01_extrair_blocos.py, linhas 37–57. Para cada bloco de texto, junta as linhas e acumula tamanho de fonte e negrito ponderados pelo nº de caracteres. Imagens (type 1) são ignoradas.

Coluna	O que guarda	Para que serve no 02
doc, pagina, bloco	plano, página (a partir de 1) e posição do bloco na página	identificação e corpo do documento
x0, y0, x1, y1	retângulo do bloco, em pontos	ordem de leitura em duas colunas; margens
larg_pag, alt_pag	tamanho da página	margem = 10% superior/inferior
tam_fonte	tamanho médio da fonte, ponderado por caracteres	tabelas (fonte pequena) e títulos (fonte grande)
negrito	fração dos caracteres em negrito	detectar títulos
n_linhas, girado	nº de linhas e fração com texto não horizontal	tabelas em paisagem
texto	as linhas do bloco, separadas por \n	tudo o resto

Duas verificações simples completam o script: blocos sem nenhum caractere visível são saltados, e se mais de 5% das páginas de um PDF não tiverem texto o script avisa que pode ser um PDF escaneado, que precisaria de OCR. Nos três planos isso não acontece.

2. 02_limpar.py — de blocos a parágrafos

Entrada: blocos_raw.parquet · **Saídas:** paragrafos.parquet (corpus limpo), descartados.parquet (o que saiu e porquê), resumo_limpeza.csv e três listas revisao_*.csv

É o script mais longo (674 linhas) porque concentra todas as decisões sobre o que conta como texto do plano. Trabalha documento a documento, em seis passos.

2.1 Normalizar caracteres e pôr em ordem de leitura

`prepara_texto` aplica a normalização Unicode NFC (e não NFKC, para não transformar “n.º” em “n.o”), desfaz ligaduras tipográficas (fi → fi) e transforma caracteres de controlo e símbolos Wingdings, que os PDFs usam como marcadores de lista, em “• ”.

Como os planos são diagramados em duas colunas, a ordem “de cima para baixo” mistura as colunas. `ordem_leitura` lê a coluna esquerda inteira e depois a direita; um bloco que atravessa o meio da página (um título largo, por exemplo) fecha a faixa e começa outra. Tabelas em paisagem, que o PyMuPDF devolve linha a linha, são reagrupadas em células por `junta_girados`.

2.2 Delimitar o corpo do documento

Capa, índice e anexos ficam de fora. As páginas de início e fim estão fixadas no dicionário `CONFIG` (ex.: PDN 2023–2027, páginas 7 a 209). Sem configuração, o script procura sozinho um bloco curto “Introdução” e um bloco “Anexos”, e imprime o que escolheu para conferência.

2.3 Descartar blocos inteiros

Primeiro calcula-se o tamanho de fonte do corpo: a mediana dos blocos com 20+ palavras. Depois, cada bloco passa por oito testes em sequência; o primeiro que se aplica é registado como motivo.

```
471      motivos = []
472      for i, r in d.iterrows():
473          t, mot = r["texto"], None
474          if r["pagina"] < inicio or r["pagina"] > fim:
475              mot = "fora_do_corpo"
476          elif em_margem.iloc[i] and eh_numero_pagina(t):
477              mot = "numero_pagina"
478          elif curto_margem.iloc[i] and r["chave"] in chaves_cab:
479              mot = "cabecalho_rodape"
480          elif r["tam_fonte"] < FONTE_PEQUENA * corpo and not eh_prosa(t.replace("\n", " ")):
481              mot = "fonte_pequena"
482          elif eh_sumario(t):
483              mot = "sumario"
484          elif eh_siglas(t):
485              mot = "siglas"
486          elif eh_legenda(t):
487              mot = "legenda"
488          elif eh_numerico(t):
489              mot = "numerico"
490          motivos.append(mot)
491      d["motivo"] = motivos
```

scripts/02_limpar.py, linhas 471–491. A cadeia de regras por bloco. A regra de fonte pequena poupa blocos que parecem prosa (eh_prosa: 12+ palavras, maioria minúsculas, quase sem números).

Motivo	Critério
fora_do_corpo	página antes do início ou depois do fim definidos
numero_pagina	na margem e só um número (arábico, romano ou “12 de 200”)
cabecalho_rodape	texto curto (≤ 12 palavras) na margem que se repete em 3+ páginas, ignorando dígitos
fonte_pequena	fonte < 85% da do corpo e não é prosa (tabelas, mapas, rótulos de gráfico)
sumario	30%+ das linhas com pontilhado “..... 12”
siglas	60%+ das linhas no formato “MAT - Ministério …”
legenda	começa com Quadro/Gráfico/Figura + identificador, ou “Fonte:” / “Nota:”
numerico	menos de 50% de letras, ou metade dos tokens são números

2.4 Montar itens e parágrafos

Dentro de cada bloco que ficou, `monta_itens` junta as linhas e abre um item novo a cada marcador (•, –) ou numeração (“7.”, “a)”, “iii)”). Itens que terminam em 3+ valores numéricos (linhas de tabela) ou que são listas de siglas separadas por vírgula também saem.

Cada item vira um parágrafo novo, um título, ou a continuação do parágrafo anterior. É título se for curto, sem pontuação final e em negrito ou fonte maior. Enunciados como “Prioridade 3.2.1: …” não contam como título, porque são conteúdo de política.

```
517              titulo = (
518                  palavras <= 15
519                  and not texto.endswith((".", ";", ","))
520                  and (r["negrito"] >= 0.6 or r["tam_fonte"] >= corpo * 1.1)
521                  and bullet != "marcador"
522                  and not ENUNCIADO.match(texto)
523              )
```

scripts/02_limpar.py, linhas 517–523. Critério de título.

Um item que começa com minúscula continua o último parágrafo inacabado (olhando até 3 parágrafos para trás, para saltar caixas de destaque). Um item que começa com maiúscula também continua se o anterior termina numa palavra de ligação (“… do” + “Plano”). Isto reconstitui frases cortadas entre colunas e páginas.

2.5 Desfazer a hifenização de fim de linha

“desen-/volvimento” deve virar “desenvolvimento”, mas “bem-/estar” e “destacando-/se” devem manter o hífen. Em vez de uma lista fixa, `decide_hifen` consulta o vocabulário dos próprios três PDFs (palavras que nunca aparecem partidas, montado por `monta_vocabulario`):

```
239  def decide_hifen(esq: str, dir_: str, vocab: Counter, simples: Counter) -> bool:
240      """True = manter o hífen ('bem-estar'); False = juntar ('desenvolvimento').
241      esq = pedaço antes do hífen; dir_ = palavra seguinte (pode ter hífen: 'se-á')."""
242      a = esq.lower()
243      b = TOKEN.match(dir_.lower())
244      if not b:
245          return False
246      b = b.group(0)
247      junto, hifen = vocab[a + b], vocab[a + "-" + b]
248      if hifen > junto:
249          return True
250      if junto > 0:
251          return False
252      if b.split("-")[0] in CLITICOS:
253          return True
254      # as duas partes são palavras que existem sozinhas: composto
255      return len(a) >= 2 and len(b) >= 2 and simples[a] > 0 and simples[b.split("-")[0]] > 0
```

scripts/02_limpar.py, linhas 239–255.

Na prática: se a forma com hífen é mais frequente nos PDFs, mantém; se a forma junta existe, junta; se a segunda parte é um pronome átono (se, lhe…), mantém; e se as duas partes existem como palavras isoladas, trata como composto. Hífens suaves (\xad) são sempre quebras de sílaba. O script juntou 212 e manteve 149 hífens; cada decisão está em `outputs/tables/revisao_hifens.csv`.

2.6 Limpezas finais

Chamadas de nota coladas às palavras (“privados5 e”) são removidas, “infra-estrutura” é padronizado para “infraestrutura” e “PND” para “PDN”. Por fim, saem parágrafos com menos de 4 palavras, rótulos curtos repetidos 3+ vezes e parágrafos idênticos repetidos 3+ vezes (fica o primeiro). Cada parágrafo recebe um `par_id` e a `secao` (o último título visto).

Quanto saiu, por motivo

Plano	fora do corpo	fonte pequena	numérico	cab./rodapé	rótulo rep.	curto	outros
PND 2013–2017	166	129	630	210	354	187	146
PDN 2018–2022	459	3.732	387	273	128	101	128
PDN 2023–2027	82	579	838	206	15	44	49

Fonte: outputs/tables/resumo_limpeza.csv. Unidades misturadas: as regras de bloco contam blocos; as de item e parágrafo contam itens. “outros” = sumário, siglas, legenda, tabela, lista de siglas, duplicado e número de página.

O PDN 2018–2022 perde 3.732 blocos por fonte pequena: é o plano com mais tabelas e quadros. Para conferir o que saiu, `revisao_descartados.csv` traz até 15 exemplos por plano e motivo, e `revisao_repetidos.csv` lista os textos que se repetem 3+ vezes.

3. 03_dicionarios.py — quanto cada plano fala de cada tema

Entrada: paragrafos.parquet + config/dicionarios.yaml · **Saídas:** dicionarios_frequencias.csv, dicionarios_diferencas.csv, dicionarios_termos.csv e validacao_<tema>.csv

3.1 Os dicionários

Cada um dos 19 temas é uma lista de expressões regulares no YAML. Os padrões cobrem as grafias antes e depois do Acordo Ortográfico de 1990 (“sec?tor” casa sector e setor) e aceitam variações (“diversifica\w*”). Um exemplo:

```
14  diversificacao:
15    rotulo: Diversificação económica
16    padroes:
17      - diversifica\w*
18      - (economia|sec?tor|sec?tores|receitas?|exportações|produtos?) não[- ]petrolífer\w*
19      - não[- ]petrolífer\w*
```

config/dicionarios.yaml, linhas 14–19.

`carrega_dicionarios` junta os padrões de um tema numa única regex, do mais longo para o mais curto, para que “gás natural” conte uma vez e não duas (“gás natural” + “gás”). Os limites impedem casar no meio de outra palavra.

```
54  def carrega_dicionarios(caminho=RAIZ / "config/dicionarios.yaml") -> dict:
55      """Junta os padrões de cada tema numa só regex. Padrões mais longos primeiro,
56      para 'gás natural' contar uma vez (e não 'gás natural' + 'gás')."""
57      dic = yaml.safe_load(Path(caminho).read_text(encoding="utf-8"))
58      out = {}
59      for tema, d in dic.items():
60          pads = sorted(d["padroes"], key=len, reverse=True)
61          alt = "|".join(f"(?:{p})" for p in pads)
62          # palavra inteira: sem letra/hífen antes nem letra depois
63          out[tema] = {
64              "rotulo": d["rotulo"],
65              "padroes": d["padroes"],
66              "regex": re.compile(rf"(?<![\w-])(?:{alt})(?!\w)", flags=re.IGNORECASE),
67          }
68      return out
```

scripts/comum.py, linhas 54–68.

3.2 Duas medidas, sem e com títulos

Para cada tema e plano, o script conta as ocorrências em cada parágrafo e calcula duas taxas, para comparar planos de tamanhos diferentes:

- **por_10mil**: ocorrências por 10 mil palavras. Mede quanto espaço o tema ocupa.
- **pct_paragrafos**: % de parágrafos que citam o tema pelo menos uma vez. Mede quão espalhado ele está, sem dar peso extra a um parágrafo que o repete muitas vezes.

Tudo é calculado duas vezes: sem títulos (versão principal) e com títulos, para ver se os títulos mudam alguma conclusão. O script imprime os temas em que a diferença é maior.

3.3 Intervalos de confiança por bootstrap

A incerteza vem da reamostragem de *parágrafos*: sorteiam-se, com reposição, tantos parágrafos quanto o plano tem, recalculam-se as duas taxas, e repete-se 2.000 vezes (B, ajustável com --B). O IC de 95% são os percentis 2,5 e 97,5. Reamostrar parágrafos, e não palavras, respeita o facto de as palavras de um mesmo parágrafo não serem independentes.

```
38  def bootstrap(occ, pal, cita, B, rng):
39      """Reamostra parágrafos com reposição; devolve as B taxas por 10 mil e % de parágrafos."""
40      n = len(occ)
41      idx = rng.integers(0, n, size=(B, n), dtype=np.int32)
42      taxa = occ[idx].sum(1) / pal[idx].sum(1) * 1e4
43      pct = cita[idx].mean(1) * 100
44      return taxa, pct
45
46
47  def ic(x):
48      return np.percentile(x, [2.5, 97.5])
```

scripts/03_dicionarios.py, linhas 38–48.

O gerador aleatório é iniciado com `[SEMENTE, índice do tema, com_titulos]`: o resultado é reprodutível e não depende da ordem dos temas no YAML.

3.4 Diferenças contra o PND 2013–2017

Para comparar cada plano de Lourenço com o de 2013, subtraem-se as 2.000 taxas reamostradas de um e do outro, par a par. A diferença reportada é a mediana dessas subtrações, com o seu IC de 95%. Uma diferença só é marcada como confiável se passar em dois testes:

```
101      # comparação com pouca informação: os dois documentos somam < 2 × MIN_OCORRENCIAS
102      occ = freq.set_index(["tema", "versao", "doc"])["ocorrencias"]
103      dif["poucas_ocorrencias"] = [
104          occ[(r.tema, r.versao, DOCS[0])] + occ[(r.tema, r.versao, r.comparacao.split(" - ")[0])]
105          < 2 * MIN_OCORRENCIAS
106          for r in dif.itertuples()
107      ]
108      # só é "confiável" se o IC exclui zero e há ocorrências suficientes
109      dif["confiavel"] = ~dif["incerta"] & ~dif["poucas_ocorrencias"]
```

scripts/03_dicionarios.py, linhas 101–109.

- **incerta**: o IC inclui zero (não dá para dizer se subiu ou desceu);
- **poucas_ocorrencias**: os dois planos somam menos de 40 ocorrências (2 × MIN_OCORRENCIAS);
- **confiavel**: nenhuma das duas.

Tema	2013	2018	2023	2023 − 2013 [IC 95%]	Confiável
Clusters	26,0	0,5	0,5	-25,4 [-30,9; -20,5]	sim
Capital humano	0,6	1,1	7,5	6,9 [4,6; 9,4]	sim
Petróleo e gás	24,9	12,2	12,9	-12,0 [-18,9; -4,6]	sim
Corrupção, transparência e prestação de contas	3,7	5,2	8,5	4,7 [1,6; 8,3]	sim
Agricultura e pecuária	29,4	25,9	23,6	-5,9 [-13,3; 1,6]	não

Ocorrências por 10 mil palavras, sem títulos. Fontes: dicionarios_frequencias.csv e dicionarios_diferencas.csv.

3.5 Que palavras casaram, e amostras para validar

`dicionarios_termos.csv` mostra, por tema e plano, cada termo encontrado e quantas vezes. É a primeira coisa a olhar quando um tema muda muito: a mudança pode vir de um único termo.

Por fim, o script sorteia até 40 parágrafos por plano entre os que citam cada tema e grava-os em `validacao_<tema>.csv`, com o trecho à volta da ocorrência e uma coluna `sentido_pretendido` para marcar à mão (s / n / ?). A proporção de “s” é a precisão do dicionário.

```
125         for doc in DOCS:
126             g = par[(par["doc"] == doc) & par["texto"].map(lambda t: bool(d["regex"].search(t)))]
127             g = g.sample(min(N_VALIDACAO, len(g)), random_state=SEMENTE + k)
128             for r in g.itertuples():
129                 ms = list(d["regex"].finditer(r.texto))
130                 m = ms[0]
131                 amostras.append({
132                     "tema": tema, "doc": doc, "par_id": r.par_id, "pagina": r.pagina,
133                     "termos": "; ".join(sorted({x.group(0).lower() for x in ms})),
134                     "trecho": "…" + r.texto[max(0, m.start() - 100): m.end() + 100] + "…",
135                     "texto": r.texto,
136                     "sentido_pretendido": "",  # marcar à mão: s / n / ?
137                     "observacao": "",
138                 })
139         pd.DataFrame(amostras).to_csv(TABELAS / f"validacao_{tema}.csv", index=False)
```

scripts/03_dicionarios.py, linhas 125–139.

> **Ainda por fazer**
> Nos 19 arquivos de validação a coluna sentido_pretendido está vazia. Enquanto não for preenchida, não se sabe a precisão de cada dicionário. Exemplo de risco: “sonangol” conta para Petróleo e gás e também para Empresas públicas.

4. 03_ngramas.py — palavras e expressões mais frequentes

Entrada: paragrafos.parquet · **Saídas:** monogramas_, bigramas_ e trigramas_<plano>.csv (contagem completa) e monogramas_top, bigramas_top, trigramas_top.csv (os 20 primeiros lado a lado)

Um n-grama é uma sequência de n palavras vizinhas: monograma (1), bigrama (2) e trigrama (3). Ao contrário dos dicionários, aqui não há lista prévia: conta-se tudo e vê-se o que aparece.

4.1 Tokenização

O texto vai para minúsculas e uma palavra é uma sequência de letras, com hífen interno permitido (“bem-estar”, “agro-indústria”). Números, pontuação e siglas com dígitos desaparecem. Títulos ficam de fora.

4.2 Primeiro forma o n-grama, depois filtra

O detalhe mais importante do script é a ordem das operações. Se as stopwords (artigos, preposições, verbos auxiliares) fossem apagadas antes, “luta contra a pobreza” viraria o bigrama artificial “luta pobreza”. Por isso forma-se o n-grama com as palavras realmente vizinhas e só então se testa:

```
67  def tokens(texto: str) -> list:
68      return PALAVRA.findall(texto.lower())
69
70
71  def ngramas(texto: str, n: int):
72      t = tokens(texto)
73      for i in range(len(t) - n + 1):
74          g = t[i : i + n]
75          if any(len(w) < 2 for w in g):
76              continue
77          # só as pontas são testadas: no bigrama isso cobre as duas palavras;
78          # no trigrama o meio pode ser preposição ("combate à pobreza")
79          if g[0] in STOPWORDS or g[-1] in STOPWORDS:
80              continue
81          yield " ".join(g)
```

scripts/03_ngramas.py, linhas 67–81.

- **monograma**: a palavra não pode ser stopword;
- **bigrama**: nenhuma das duas pode ser stopword (“ensino superior”);
- **trigrama**: só as pontas são testadas, o meio pode ser preposição (“abastecimento de água”, “combate à pobreza”);
- palavras de uma letra são descartadas e nenhum n-grama atravessa parágrafos.

4.3 Normalizar pelo tamanho

`por_10mil` divide cada contagem pelo total de n-gramas do mesmo tipo no plano. Como o PDN 2018–2022 tem quase o dobro do texto, as contagens brutas (coluna n) não são comparáveis entre planos; as taxas são.

Os cinco bigramas e trigramas mais frequentes

#	PND 2013–2017	PDN 2018–2022	PDN 2023–2027
1	ensino superior (92)	recursos humanos (68)	capital humano (45)
2	recursos humanos (61)	ensino superior (64)	sistema nacional (39)
3	projectos estruturantes (57)	formação profissional (62)	prioridade desenvolvimento (33)
4	sistema nacional (43)	administração pública (59)	longo prazo (32)
5	formação profissional (40)	administração local (52)	administração pública (29)
#	**PND 2013–2017**	**PDN 2018–2022**	**PDN 2023–2027**
1	instituições de ensino (30)	qualidade de vida (44)	ministério de tutela (16)
2	sector empresarial público (29)	abastecimento de água (43)	serviços de saúde (15)
3	diversificação da economia (26)	mercado de trabalho (35)	ambiente de negócios (14)
4	nacional de desenvolvimento (25)	aumenta em relação (32)	cadeia de valor (14)
5	abastecimento de água (23)	local do estado (32)	igualdade de género (14)

Contagem bruta entre parênteses. Fontes: bigramas_top.csv e trigramas_top.csv.

A tabela já mostra a mudança de vocabulário: “projectos estruturantes” e “sector empresarial público” em 2013; “capital humano” e “ministério de tutela” em 2023. Mas também mostra dois artefactos, descritos a seguir.

5. Pontos de atenção

Ao ler o código encontrei questões que não estão erradas, mas que afetam a leitura dos resultados ou a manutenção. Estão por ordem de impacto.

- **Números apagados juntam palavras que não eram vizinhas.** A tokenização remove “9.1.2:”, então “Prioridade 9.1.2: Desenvolvimento da rede urbana” gera o bigrama “prioridade desenvolvimento”, 3.º do PDN 2023–2027 (33 vezes). Solução: partir o texto também onde havia número ou pontuação forte.
- **Fórmulas de template entram nos n-gramas.** “meta” (529) e “objectivo” (375) estão entre os monogramas de topo de 2018, e “aumenta em relação” é o 4.º trigrama, vindos das fichas de metas. O 04_distintivos exclui esse vocabulário (config/template.yaml); o 03_ngramas não.
- **Código duplicado.** O 03_ngramas tem a sua própria cópia de STOPWORDS, tokens() e ngramas(). Hoje são iguais às do comum.py; se uma mudar, os n-gramas deixam de bater com os scripts 04 a 06. Melhor importar do comum.py.
- **Validação pendente.** Ver secção 3.5: as frequências dos temas ainda não têm precisão medida.
- **“PND” vira “PDN” no texto.** A substituição em 02_limpar.py (linha 67) uniformiza a sigla, mas apaga a diferença de nome do plano de 2013 em qualquer análise que procure a sigla.
- **Dois scripts com o número 03.** São independentes e a ordem entre eles não importa, mas o nome sugere uma sequência. Vale a pena dizê-lo no README ou renumerar.
- **Unidades misturadas no resumo da limpeza.** As colunas desc_* somam blocos e itens/parágrafos. Não são comparáveis entre motivos de níveis diferentes.